Porovnanie trénovania CNN od nuly a pomocou transfer learningu

Obsah

Úloha 8: CNN od nuly vs. transfer learning **2**
Cieľ úlohy . . . 2
Definícia úlohy . . . 2
Postup riešenia . . . 2

Odovzdanie a hodnotenie **4**
Dokumentácia výsledkov (PDF) . . . 4
Obhajoba riešenia . . . 4

Odporúčané nástroje a technická realizácia **6**
Matlab . . . 6
Python . . . 6

Teoretické poznámky **7**
Trénovanie od nuly a transfer learning . . . 7
Fine-tuning a jeho vzťah k transfer learningu . . . 7
Prečo sa pri CNN používa predtrénovaný model . . . 7
Augmentácia . . . 7

Ukážky minimálneho rozsahu tabuliek v dokumentácii **9**

Úloha 8: Porovnanie trénovania CNN od nuly a pomocou transfer learningu (5b)

Cieľ úlohy

Porovnajte trénovanie konvolučných neurónových sietí **od nuly** (*from scratch*) a **transfer learningom** (*TL*) na datasete **Food101**. Cieľom je overiť, ako sa pri rovnakom probléme mení úspešnosť klasifikácie, stabilita výsledkov a priebeh učenia pri rôznych architektúrach a pri dvoch odlišných prístupoch k inicializácii váh.

Porovnávajte tri modely:

1. **M1**: **AlexNet alebo VGG16**
2. **M2**: **ResNet18 alebo ResNet34**
3. **M3**: **MobileNet alebo EfficientNet**

Pre každý model porovnajte trénovanie **od nuly** (*from scratch*) a **transfer learningom** (*TL*). Každú z oboch variant **spustite 3-krát odznova** a výsledky štatisticky vyhodnoťte. Aspoň **jedna** z porovnávaných štruktúr musí na testovacích dátach dosiahnuť **priemernú úspešnosť aspoň 90% bez augmentácie, pričom po doplnení augmentácie musí dosiahnuť priemernú úspešnosť aspoň 93%**, pričom priemer sa určí z príslušných **3 behov**.

Definícia úlohy

Na riešenie použite dataset **Food101**. Ten istý dataset sa používa pri riešení v **Matlabe aj Pythone**.

Pri **Pythone** použite dataset načítaný priamo cez `torchvision.datasets.Food101`. Pri **Matlabe** použite pripravený kód zo štandardného repozitára `UMINT-UNS_data`, v ktorom je pre Zadanie 8 uvedený postup, ako dataset **Food101** stiahnuť a načítať do Matlabu.

Z celého datasetu použite iba **10** vybraných tried:

```
apple_pie, caesar_salad, clam_chowder, edamame, french_fries, hamburger, hot_dog,
ice_cream, sushi, waffles
```

Rovnaký výber tried musí byť použitý pri všetkých modeloch, vo všetkých behoch a pri oboch režimoch trénovania.

Dataset **Food101** má pevne definované rozdelenie na **trénovaciu** a **testovaciu** časť. Validačnú množinu vytvorte dodatočne z trénovacej časti.

Pri variante **transfer learningom** (*TL*) použite predtrénované váhy z **ImageNetu**. Pri variante **od nuly** (*from scratch*) inicializujte sieť bez predtrénovaných váh a trénujte ju od začiatku.

Postup riešenia

1. Použite dataset **Food101** a z neho vyberte iba **10** tried:

   ```
   apple_pie, caesar_salad, clam_chowder, edamame, french_fries, hamburger, hot_dog,
   ice_cream, sushi, waffles
   ```

   Z pôvodnej trénovacej časti datasetu vytvorte **trénovaciu** a **validačnú** množinu. Testovaciu časť ponechajte oddelenú na finálne vyhodnotenie.

2. Pri riešení v **Matlabe aj Pythone** používajte ako základ ukážkové kódy z predchádzajúcich zadaní a tie ďalej upravujte pre potreby tejto úlohy. **AI slop** alebo iný cudzí kód sa považuje za **plagiát**.

3. Zvoľte tri modely **M1**, **M2**, **M3** podľa zadania. Pri **M1** si vyberte **AlexNet alebo VGG16**, pri **M2** použite **ResNet18 alebo ResNet34** a pri **M3** si vyberte **MobileNet alebo EfficientNet**.

4. Pre **každý** model vykonajte dve varianty trénovania: **od nuly** (*from scratch*) a **transfer learningom** (*TL*). Každú z týchto dvoch variant **spustite 3-krát odznova**. Pre každý model tak ide celkovo o **6 spustení**, teda pre tri modely spolu **18 spustení**.

5. Pri porovnávaní modelov zachovajte **rovnaké hlavné hyperparametre** trénovania, najmä počet epoch, learning rate, batch size a ostatné hlavné nastavenia. Výnimkou môže byť iba **early stopping**, ak ho v riešení použijete.

6. Po základnom porovnaní modelov vyberte **najlepší model bez augmentácie**. Pre tento model následne vykonajte **3 nové spustenia trénovania s augmentáciou dát** a porovnajte ich s predchádzajúcimi výsledkami toho istého modelu bez augmentácie. Vyhodnoťte, ako augmentácia ovplyvnila priebeh učenia, pretrénovanie a finálne výsledky.

7. Pre **najlepší výsledok z každého modelu** zdokumentujte priebeh učenia, dosiahnuté úspešnosti klasifikácie a niekoľko ukážok predikcií, teda obrázok, skutočnú triedu a predikovanú triedu. Následne vykonajte **porovnanie od nuly** (*from scratch*) **vs. transfer learningom** (*TL*) pre modely **M1**, **M2**, **M3** a samostatne aj **porovnanie najlepšieho modelu bez augmentácie a s augmentáciou**.

Poznámka: Pri porovnávaní modelov je cieľom porovnávať **porovnateľné nastavenia**. Ak sa mení architektúra alebo typ inicializácie, ostatné hlavné hyperparametre by mali zostať čo najviac rovnaké, aby bolo možné výsledky férovo porovnať. Ak používate validačné dáta, graf **loss vs. epoch** zobrazujte ako spoločný graf trénovacej a validačnej chyby. Na testovacích dátach sa potom uvádzajú už len výsledné metriky po natrénovaní siete.

Podobne ako v predchádzajúcich zadaniach, všetky priebehy učenia a porovnania musia byť uvedené ako **vlastné grafy alebo figúry** s popisom osí a legendou. **Screenshoty z TensorBoardu alebo z iných nástrojov sa nepovažujú za dostatočnú formu spracovania výsledkov.**

Zároveň **musia byť** výsledky v dokumentácii spracované nielen vo forme grafov, ale aj v **prehľadných tabuľkách**. Najmä použité hyperparametre, porovnávané modely, výsledky jednotlivých behov a porovnanie *from scratch* vs. *TL* **musia byť** uvedené tabuľkovo, aby bolo porovnanie čitateľné, jednoznačné a malo primeranú technickú formu. Ukážky minimálneho rozsahu a formy tabuliek, ktoré majú byť súčasťou dokumentácie, sú uvedené na konci dokumentu v Tab. 1, Tab. 2, Tab. 3, Tab. 4, Tab. 5, Tab. 6, Tab. 7 a Tab. 8.

Odovzdanie a hodnotenie

Hodnotenie zadania prebieha formou osobnej konzultácie. Zadanie sa odovzdáva vždy na **začiatku cvičenia** po odprezentovaní nasledujúceho zadania.

Počas semestra môže študent nahradiť najviac dve cvičenia. V takom prípade je možné zadanie odovzdať aj na cvičení, na ktorom študent nahrádza. V ostatných prípadoch sa zadanie odovzdáva vždy svojmu cvičiacemu.

Všetky požadované výsledky musia byť pripravené **už na začiatku cvičenia** a uvedené v dokumentácii. Opätovné spúšťanie trénovania počas obhajoby sa vykonáva len v prípade, že o to cvičiaci výslovne požiada.

Dokumentácia výsledkov (PDF)

Pripravte stručnú správu (PDF), ktorá obsahuje grafy z viacerých spustení a krátku analýzu vplyvu zvolených parametrov. Pri neurónových sieťach typicky uvádzajte priebeh učenia ako **loss vs. epoch** a podľa typu úlohy aj vhodné metriky. Dokument slúži ako podklad pre diskusiu, aby nebolo nutné čakať na opakované výpočty počas obhajoby.
Pre toto zadanie uveďte v správe:

1. stručný opis úlohy, použitých dát a spôsobu práce s rozdelením **train/validation/test**,
2. opis zvolených modelov **M1**, **M2**, **M3** a použitých hyperparametrov,
3. pre **každý** model výsledky **3 spustení od nuly** (*from scratch*) a **3 spustení transfer learningom** (*TL*), vrátane minimálnej, maximálnej a priemernej úspešnosti klasifikácie,
4. pre **najlepší výsledok z každého modelu a režimu** uveďte priebeh učenia a dosiahnuté úspešnosti na trénovacích, validačných a testovacích dátach,
5. samostatné porovnanie **od nuly** (*from scratch*) **vs. transfer learningom** (*TL*) pre modely **M1**, **M2**, **M3**,
6. pre **najlepší model bez augmentácie** uveďte jeho výsledky a následne porovnanie s tým istým modelom po **3 behoch s augmentáciou dát**, vrátane vplyvu augmentácie na priebeh učenia, pretrénovanie a finálne výsledky,
7. ukážku vybraných predikcií pre najlepšie modely, teda obrázok, skutočnú triedu a predikovanú triedu.

Súčasťou dokumentácie **musia byť aj prehľadné tabuľky** so zvolenými hyperparametrami, porovnávanými modelmi a výsledkami jednotlivých behov.

Ukážky minimálneho rozsahu a formy tabuliek sú uvedené na konci dokumentu v Tab. 1, Tab. 2, Tab. 3, Tab. 4, Tab. 5, Tab. 6, Tab. 7 a Tab. 8.

Obhajoba riešenia

Súčasťou hodnotenia je demonštrácia funkčného kódu a vysvetlenie implementácie. Body sa udeľujú najmä za porozumenie princípu algoritmu a schopnosť zdôvodniť dosiahnuté výsledky, nie iba za spustiteľný program.

V diskusii buďte pripravení interpretovať grafy učenia (**loss vs. epoch** a prípadné metriky) a zdôvodniť vplyv nastavení (napr. architektúra, regularizácia, learning rate, batch size) na konvergenciu a opakovateľnosť výsledkov.

Pri obhajobe vysvetlite, aké vstupné a výstupné dáta boli použité, ktoré modely boli zvolené ako **M1**, **M2**, **M3**, aké parametre trénovania boli nastavené a v čom sa porovnávané architektúry odlišujú. Na základe grafov učenia a dosiahnutých výsledkov zdôvodnite, prečo bol vybraný najlepší model a aké rozdiely sa prejavili medzi trénovaním **od nuly** (*from scratch*) a **transfer learningom** (*TL*).

Zároveň vysvetlite, prečo sa pri porovnávaní modelov používajú **opakované spustenia trénovania**, prečo boli zvolené práve vybrané triedy a ako sa augmentácia prejavila na pretrénovaní a na výslednej úspešnosti klasifikácie.

Súčasťou obhajoby je aj preukázanie porozumenia vlastnému kódu v **Matlabe aj Pythone**. **AI slop** alebo iný cudzí kód sa považuje za **plagiát**.

Odporúčané nástroje a technická realizácia

Zadanie je možné vypracovať v prostredí **Matlab** alebo v jazyku **Python**. Ako alternatívu bez inštalácie je možné použiť **MATLAB Online** (`https://matlab.mathworks.com/`). Pre Python odporúčame **Google Colab** (`https://colab.research.google.com/`).

Matlab

Pri Matlabe použite vhodné nástroje pre neurónové siete podľa pokynov na cvičení. Do správy zahrňte priebeh učenia (loss vs. epoch) a výsledné metriky podľa zadania.

Pri Matlabe použite štandardný repozitár `UMINT-UNS_data`, v ktorom je pre Zadanie 8 pripravený kód na stiahnutie a načítanie datasetu **Food101**.

Použite iba vybrané triedy uvedené vyššie v zadaní. Rovnaký výber tried musí byť použitý pri všetkých modeloch a vo všetkých behoch.

Na načítanie obrazových dát použite vhodný postup podľa poskytnutého kódu, typicky napr. `imageDatastore`. Pre transfer learning použite predtrénované siete dostupné v Matlabe, napr. `alexnet`, `vgg16`, `resnet18`, prípadne `resnet34` ak je v danom prostredí dostupný, `mobilenetv2` alebo `efficientnet`.

Python

Pri Pythone odporúčame pracovať v prostredí Google Colab. Pre úlohy v tomto bloku použite oficiálne repozitáre:

- **Dodatočné kódy a zadania:** `https://github.com/STU-FEI-OUI/UMINT-UNS`
- **Dáta pre Python:** `https://github.com/STU-FEI-OUI/UMINT-UNS_data`

Pre vizualizáciu uveďte aspoň priebeh učenia ako **loss vs. epoch** a podľa zadania doplňte validáciu a metriky.

Pri Pythone použite dataset `Food101` priamo cez `torchvision.datasets.Food101`.

```
from torchvision import datasets, transforms
transform = transforms.Compose([transforms.Resize((224,224)), transforms.ToTensor()])
train_dataset = datasets.Food101(root="./data", split="train", download=True, transform=transform)
test_dataset = datasets.Food101(root="./data", split="test", download=True, transform=transform)
```

Použite iba vybrané triedy uvedené vyššie v zadaní. Dataset po načítaní prefiltrujte tak, aby všetky modely používali rovnakú podmnožinu tried. Na vytvorenie dávok použite napr. `DataLoader`. Pri Pythone môžete použiť napríklad tieto modely:

```
models.alexnet
models.vgg16
models.resnet18
models.resnet34
models.mobilenet_v2
models.efficientnet_b0 (alebo iná ver.)
```

Pri *TL* použite predtrénované váhy, pri variante *from scratch* nastavte `weights=None`.

Teoretické poznámky

Trénovanie od nuly a transfer learning

Pri **trénovaní od nuly** (*from scratch*) sa celá sieť inicializuje bez predtrénovaných váh a všetky jej parametre sa učia až z nových dát. Takýto prístup dáva modelu najväčšiu voľnosť, ale zvyčajne vyžaduje viac dát, viac epoch a citlivejšie ladenie hyperparametrov. Ak dataset nie je dostatočne veľký alebo dostatočne rozmanitý, model trénovaný od nuly sa môže učiť pomalšie a menej stabilne.

Pri **transfer learningu** sa naopak využije už existujúci predtrénovaný model, typicky natrénovaný na datasete **ImageNet**. V takomto modeli je už **extrakčná časť CNN siete** naučená rozpoznávať všeobecné obrazové príznaky, napríklad hrany, textúry, jednoduché tvary a ich kombinácie. Pri novej úlohe sa potom spravidla ponechá táto extrakčná časť modelu a upraví sa najmä **klasifikačná časť**, ktorá sa prispôsobí novému počtu tried a novej úlohe.

V praxi sa dnes pri obrazových úlohách veľmi často používa práve **transfer learning**, pretože vedie k rýchlejšiemu trénovaniu, stabilnejšej konvergencii a často aj k lepšej generalizácii. Ak nový dataset nie je veľmi veľký, býva tento prístup spravidla výhodnejší než úplné trénovanie od nuly.

Fine-tuning a jeho vzťah k transfer learningu

Pojmy **transfer learning** a **fine-tuning** sa v praxi niekedy používajú voľnejšie, ale z technického hľadiska je vhodné ich rozlišovať.

Ako **transfer learning** budeme v tomto texte chápať situáciu, keď sa využije predtrénovaný model, **extrakčná časť CNN siete zostane zamrazená** a učí sa len **klasifikačná časť**. Tento prístup je vhodný najmä vtedy, keď je nová úloha stále pomerne blízka pôvodnej oblasti, na ktorej bol model natrénovaný. Predtrénovaný extraktor príznakov vtedy už poskytuje dostatočne užitočné reprezentácie a stačí prispôsobiť len klasifikátor na nové triedy.

Ako **fine-tuning** budeme chápať situáciu, keď sa po inicializácii predtrénovanými váhami neučí len klasifikačná časť, ale odomkne sa aj časť **extrakčnej časti siete**, prípadne celý model. Takéto doladenie sa zvyčajne robí s **malým learning rate**, aby sa pôvodne naučené príznaky neprepísali príliš agresívne, ale aby sa jemne prispôsobili novej úlohe.

Ak je nový problém vizuálne podobný pôvodnej oblasti, často stačí čistý transfer learning. Ak je však rozdiel medzi pôvodnými a novými dátami väčší, býva vhodnejší fine-tuning. Napríklad pri prechode z bežných fotografií na špecializované medicínske alebo technické obrazové dáta už samotná výmena klasifikačnej časti často nestačí a je potrebné doladiť aj extrakčnú časť modelu. V tomto zadaní sa však porovnáva len **trénovanie od nuly** a **transfer learning**, nie plný fine-tuning celého modelu.

Prečo sa pri CNN používa predtrénovaný model

Pri CNN sieťach má predtrénovaný model veľký význam preto, že **extrakčná časť siete** sa učí všeobecné obrazové príznaky, ktoré sú do istej miery prenositeľné medzi rôznymi úlohami. Ak už model vie rozpoznávať základné vizuálne štruktúry, nová úloha nemusí začínať úplne od nuly. To je hlavný dôvod, prečo transfer learning a fine-tuning v obrazových úlohách tak často fungujú.

Na druhej strane však neplatí, že predtrénovaný model bude automaticky najlepší vždy. Ak je nový dataset veľmi špecifický, veľmi odlišný od ImageNetu, alebo ak je k dispozícii veľké množstvo vlastných dát, môže mať význam aj trénovanie od nuly. Toto zadanie preto nepredpokladá jeden univerzálne správny výsledok, ale vedie k priamemu porovnaniu oboch prístupov na rovnakom probléme.

Augmentácia

Pri obrazových úlohách sa často používa **augmentácia dát**, teda vytváranie pozmenených verzií tréningových obrázkov, napríklad pomocou orezania, preklopenia, rotácie, posunu alebo zmeny farieb. Cieľom je zvýšiť rozmanitosť tréningových dát bez potreby zbierať nové vzorky.

Augmentácia môže pomôcť obmedziť pretrénovanie a zlepšiť generalizáciu modelu. Na druhej strane však môže predĺžiť trénovanie a pri nevhodnom nastavení môže aj zhoršiť výsledky. Preto má zmysel porovnávať ten istý model bez augmentácie a s augmentáciou, nie iba porovnať dva odlišné modely medzi sebou.

Ukážky minimálneho rozsahu tabuliek v dokumentácii

Nasledujúce tabuľky predstavujú **minimálny vzor**, ako má byť dokumentácia štruktúrovaná. Ukážky sú vypísané pre **model M1**. **Ekvivalentné tabuľky rovnakého typu** majú byť v dokumentácii vytvorené aj pre **M2** a **M3**. Vo všetkých tabuľkách s metrikami platí, že ak sa uvádza **accuracy**, musí byť spolu s ňou uvedený aj príslušný **loss**.

Model	Architektúra	Epochy	LR	Batch
M1	VGG16	20	0.0001	128
M2	ResNet18	20	0.0001	128
M3	MobileNet	20	0.0001	128

Tabuľka 1: Zvolené modely a hlavné hyperparametre.

Beh	Train loss	Train acc [%]	Val loss	Val acc [%]	Test loss	Test acc [%]
1	0.412	86.2	0.536	83.8	0.548	84.7
2	0.398	86.9	0.521	84.4	0.531	85.1
3	0.405	86.5	0.529	84.0	0.540	84.9

Tabuľka 2: Výsledky 3 behov pre model M1 pri trénovaní od nuly (*from scratch*). Rovnaký typ tabuľky má byť uvedený aj pre modely M2 a M3.

Beh	Train loss	Train acc [%]	Val loss	Val acc [%]	Test loss	Test acc [%]
1	0.128	95.1	0.174	93.2	0.181	93.8
2	0.121	95.4	0.169	93.7	0.176	94.1
3	0.125	95.2	0.171	93.5	0.178	94.0

Tabuľka 3: Výsledky 3 behov pre model M1 pri transfer learningu (*TL*). Rovnaký typ tabuľky má byť uvedený aj pre modely M2 a M3.

Režim	Priemer train loss	Priemer train acc [%]	Priemer val loss	Priemer val acc [%]	Priemer test loss	Priemer test acc [%]
scratch	0.405	86.5	0.529	84.1	0.540	84.9
TL	0.125	95.2	0.171	93.5	0.178	94.0

Tabuľka 4: Súhrnné porovnanie režimov trénovania od nuly (*from scratch*) a transfer learningu (*TL*) pre model M1. Rovnaký typ tabuľky má byť uvedený aj pre modely M2 a M3.

Model	Architektúra	Priemer test loss od nuly	Priemer test acc [%] od nuly	Splnenie podmienky 90/%
M1	VGG16	0.540	84.9	nie
M2	ResNet18	0.455	88.8	nie
M3	MobileNet	0.366	91.7	nie

Tabuľka 5: Súhrnné porovnanie priemerných výsledkov modelov M1, M2, M3 pri trénovaní od nuly (*from scratch*).

Model	Architektúra	Priemer test loss TL	Priemer test acc [%] TL	Splnenie podmienky 93%
M1	VGG16	0.178	94.0	nie
M2	ResNet18	0.124	96.9	áno
M3	MobileNet	0.071	98.8	áno

Tabuľka 6: Súhrnné porovnanie priemerných výsledkov modelov M1, M2, M3 pri transfer learningu (*TL*).

Beh	Train loss	Train acc [%]	Val loss	Val acc [%]	Test loss	Test acc [%]
1	0.109	95.8	0.151	94.2	0.159	94.7
2	0.104	96.0	0.147	94.5	0.154	94.9
3	0.107	95.9	0.149	94.3	0.157	94.8

Tabuľka 7: Výsledky 3 behov pre najlepší model po doplnení augmentácie.

Varianta	Val loss	Val acc [%]	Test loss	Test acc [%]	Priemer epochy začiatku pretrénovania
bez augmentácie	0.171	93.5	0.178	94.0	8.7
s augmentáciou	0.149	94.3	0.157	94.8	12.3

Tabuľka 8: Porovnanie najlepšieho modelu bez augmentácie a s augmentáciou.